

UNIVERSIDAD MARIANO GALVEZ
FACULTAD: DE INGENIERÍA
CARRERA: INGENIERÍA EN SISTEMAS Y CIENCIAS DE LA COMPUTACIÓN
Curso: PROGRAMCION II
Sección: B
Catedrático: **MOISES EDUARDO VELASQUEZ OLIVA**

PROYECTO

Carrito GT

Estudiante: HENRY HOMAR TORRES FRANCO
Carné No.: 3190-24-13372
Fecha: 17-10-2026

INDICE

Tema	No. De Pagina
Introducción	1
Contenido	2
Infraestructura y Entorno de Desarrollo	2
Arquitectura del Sistema	2
Diagrama de casos de Uso	2
Diagrama de Clases	4
Arquitectura de Capas del Backend en Spring Boot	4
Implementación de Pruebas Automatizadas	6
Estrategia de Pruebas con Mockito	6
Explicaciones más relevantes	7
Manual del Usuario	8
Conclusión	13
E-grafía/Referencias	14

Introducción

El desarrollo del presente proyecto es el impulso hacia el aprendizaje de tecnologías modernas para el desarrollo de aplicaciones y software, el aprendizaje implica aprender intuitivamente por nuestra propia cuenta el desarrollo del mismo, los obstáculos a lo largo de este proyecto solo implican el descubrimiento del conocimiento requerido para comprender el código a lo largo del desarrollo del software.

El presente documento detalla el proceso de migración y modernización de un sistema de carrito de compras originalmente desarrollado en PHP, el cual fue brindado por el propio curso de Programación II. El objetivo principal de este proyecto según el enunciado proporcionado es implementar una arquitectura de 6 capas basada en el patrón Modelo-Vista-Controlador (MVC). Para lograr esto, se separó el sistema monolítico en dos aplicaciones independientes: un backend desarrollado con Spring Boot de Java, que expone una API RESTful, desarrollado en el IDE de Intellij IDEA, y un frontend moderno y reactivo construido con Angular, desarrollado en Visual Estudio Code. Toda la infraestructura de desarrollo, bases de datos y despliegue se configuró sobre un entorno virtualizado en Linux, garantizando un ecosistema aislado, control de recursos estandarizado y resolución de políticas de seguridad compartidas (CORS).

Contenido

Infraestructura y Entorno de Desarrollo

El sistema fue desplegado y probado en un entorno de Ubuntu Linux virtualizado mediante VMware Workstation, elegí VMware porque es una máquina que gestiona el funcionamiento de recursos compartidos tomando en cuenta los recursos físicos de la computadora y busca un balance de funcionalidad a modo que no afecte el sistema operativo anfitrión. Luego se aumentó los recursos de RAM, Procesador y espacio en Disco. Se gestionó cuidadosamente las asignaciones correspondientes antes mencionadas para soportar la ejecución concurrente de los siguientes servicios:

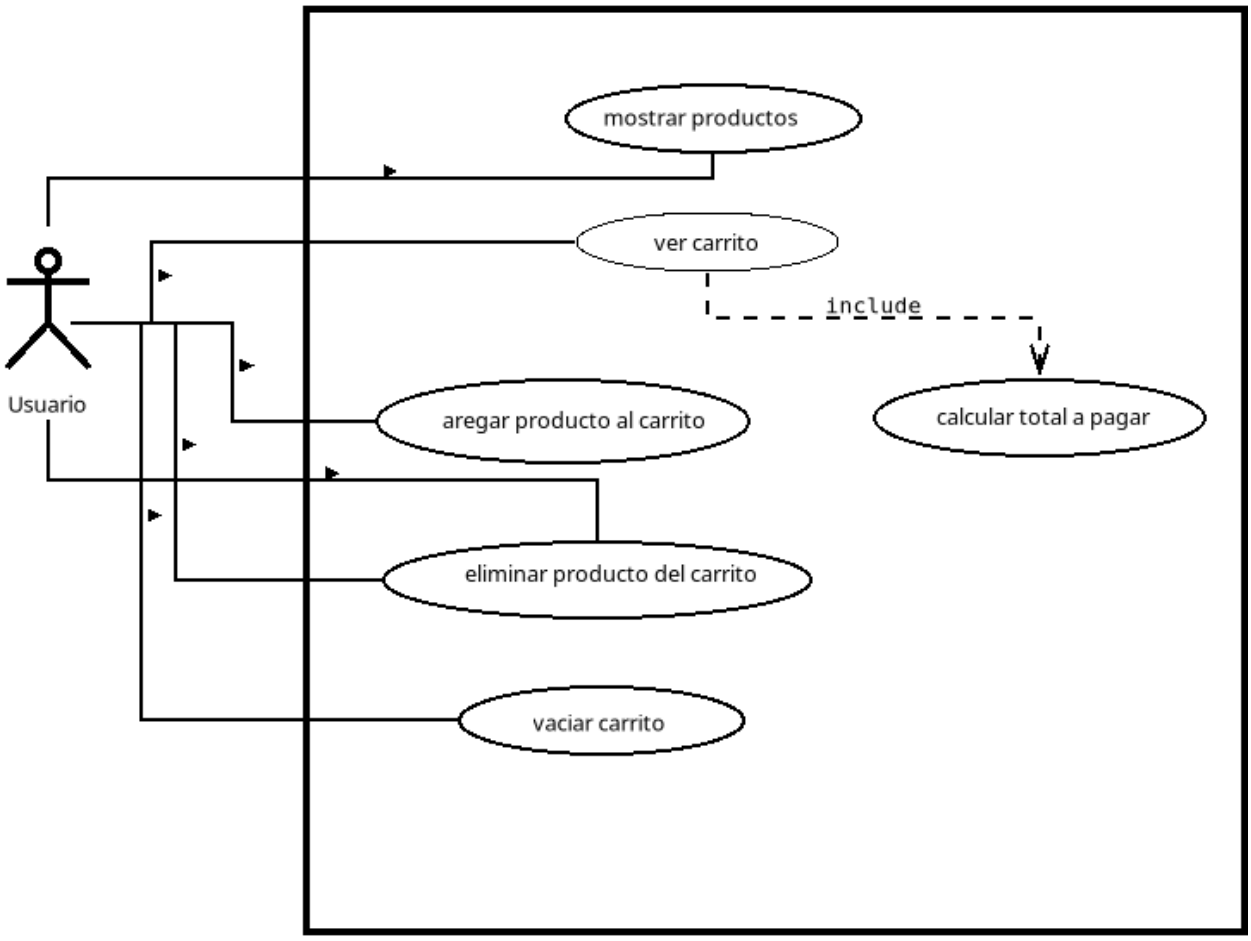
Base de Datos en MySQL Server para la persistencia relacional de los productos y el carrito. El Backend que es la capa de lógica y datos, desarrollado en Spring Boot ejecutándose en el puerto 8080. El Frontend, este es la capa de presentación desarrollada en Angular ejecutándose en el puerto 4200, consumiendo la API de Spring Boot mediante peticiones HTTP asíncronas.

El entorno original de este proyecto fue en base del lenguaje de PHP, este fue, lo principal para comprender el punto de partida de la migración, se analizó el código original en PHP. Este enfoque clásico requería un stack tradicional (como XAMPP) para interpretar los scripts y servir las vistas dinámicas directamente desde el servidor. Este modelo acoplado fue reemplazado exitosamente por la actual separación estricta entre cliente y servidor, mencionado en el documento del enunciado del proyecto.

Arquitectura del Sistema

Diagrama de Casos de Uso

El diagrama de casos de uso modela las interacciones principales entre el Usuario que es el Actor en la herramienta de desarrollo DIA, y que asemeja lo desarrollado en la aplicación del frontend construida en Angular. Se definieron los siguientes flujos principales de uso para el usuario:



i. Visualizar Catálogo

El usuario puede cargar y ver la lista completa de productos disponibles, “mostrar productos” consumiendo el método GET del backend.

ii. Gestión del Carrito

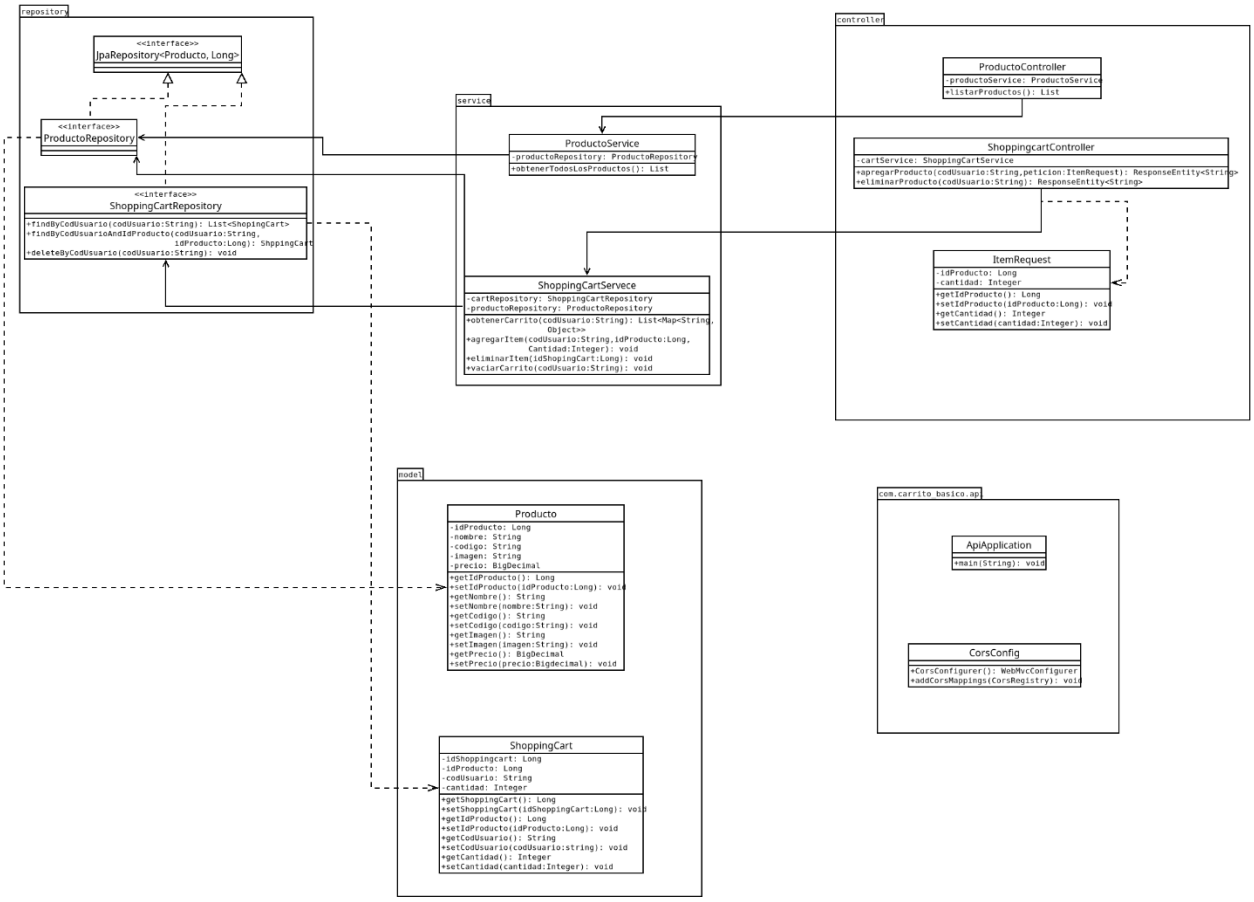
Incluye las acciones de "agregar producto al carrito", "ver mi carrito", “eliminar producto del carrito” y "vaciar carrito". Al vaciar el carrito muestra el dialogo de carrito vaciado y al dar en ok, devuelve al cátalo de productos.

iii. Cálculo de la selección de productos

Se incluyó una relación de inclusión (<<include>>) al momento de "ver carrito", ya que el sistema calcula automáticamente los subtotales y el gran total a pagar procesando el JSON devuelto por la API, sin la necesidad de recargar la página.

Diagrama de Clases

El diagrama de clases desarrollado en el programa de desarrollo de UML, DIA, este diagrama de clases representa la estructuración del desarrollo del api el backend, desarrollado en Spring Boot en Java, y realizado en el IDE de Intellij DIA, el diseño del backend en Spring Boot se estructuró siguiendo una estricta separación de responsabilidades para cumplir con el patrón MVC y el diseño multicapas, lo que representa este diseño de multicapas es garantizar que el código sea escalable y fácil de mantener.



Arquitectura de Capas del Backend en Spring Boot

El diseño del sistema se fundamenta en una arquitectura de capas estricta, asegurando la separación de responsabilidades, esto para que su comprensión tanto de diseño de clases como de código sea más fácil su comprensión. Cada petición que el Frontend ósea en nuestro caso Angular envía atraviesa este flujo estructurado antes de tocar la base de datos de MySQL. A continuación se definen las capas acompañadas de una breve explicación.

1. Capa de Presentación / Web (Controladores - Controllers)

La función principal de la capa controller es la puerta de entrada de la API RESTful, en Spring Boot, su única responsabilidad es interceptar las peticiones HTTP (GET, POST, DELETE) provenientes del navegador, interpretar la URL y recibir los datos en formato JSON.

A demás en su implementación técnica se utilizaron clases como ShoppingCartController anotadas con @RestController. Esta capa no realiza cálculos matemáticos ni consultas a la base de datos; simplemente toma el JSON entrante, lo convierte en objetos Java temporales, esto lo logra utilizando el patrón DTO - *Data Transfer Object*, que a su vez es implementada con la clase ItemRequest, y le delega el trabajo pesado a la capa de servicios. Al finalizar, empaqueta la respuesta en códigos HTTP 200 OK que significa que fue aceptado o accedido sin problemas o bien el código 404, este implica que hubo un error al renderizar la petición o también podría ser un problema al convertir el JSON en objetos y por eso lanzaría el 404 Not Found y se la devuelve a Angular, y si se observa en la consola del navegador mostrara este código.

2. Capa de Lógica de Negocio, Servicios - Services

Su función principal para esta capa “service”, es de ser el "cerebro" de la aplicación. Aquí reside toda la lógica empresarial, las reglas del negocio y las validaciones.

Su implementación técnica por las clases como ShoppingCartService, y ProductoService, están inyectadas en los controladores. Si un usuario quiere agregar un producto al carrito, esta capa recibe la orden del controlador, verifica qué producto es, calcula si debe sumar una nueva cantidad o crear un nuevo registro, y coordina las acciones necesarias. Aislar esta lógica aquí garantiza que, si el día de mañana se cambia Angular por una aplicación móvil, la lógica de negocio seguirá intacta y pueda seguir utilizándose sin reescribir código.

3. Capa de Acceso a Datos, Repositorios - DAO

Esta capa es la única capa autorizada para comunicarse directamente con el motor de base de datos MySQL. Implementa el patrón DAO (*Data Access Object*), el cual aísla al resto del sistema de la complejidad de las consultas SQL.

Su implementación técnica fue construir interfaces como ShoppingCartRepository que heredan de JpaRepository<T, ID>. Al utilizar el framework **Spring Data JPA**, la aplicación genera dinámicamente las consultas de base de datos en tiempo de ejecución. Esto permite utilizar métodos como findByCodUsuario o deleteByCodUsuario de forma nativa, aplicando la persistencia de datos de manera segura y mitigando vulnerabilidades críticas como la Inyección SQL.

4. Capa de Dominio (Modelos / Entidades - Entities)

Su función principal es la representación de la estructura de los datos del mundo real dentro del código. Dicho en otras palabras, son el molde de la información.

Su implementación técnica es de las clases como Producto y ShoppingCart que actúan como entidades de reconocimiento. A través del mapeo objeto-relacional (ORM) provisto por Hibernate/JPA, cada clase Java se mapea automáticamente a una tabla en MySQL, y cada atributo (como idProducto, nombre, precio) se convierte en una columna de la base de datos.

5. Capa Transversal de Seguridad y Configuración (Config)

Su función principal es definir las reglas globales que afectan a toda la aplicación sin interferir con la lógica de negocio.

Se implementó la clase CorsConfig para gestionar las políticas de intercambio de recursos de origen cruzado (CORS). Esta capa actúa como un filtro de seguridad en el servidor que autoriza explícitamente las transacciones provenientes del puerto 4200 de Angular, permitiendo el flujo bidireccional seguro de credenciales y cabeceras HTTP.

Nota: se configuro la clase CORS como CorsConfig global porque @Cors de Spring Boot nos bloqueaba muchas peticiones, o peticiones secundarias a un GET, o a veces las peticiones secundarias de los POST, y esto es debido a la seguridad que maneja Spring Boot en su @cors, por lo tanto se uso el DTO ItemRequest en el controlador para recibir el JSON, y con la intención de conectar los repositorios y que sus peticiones ya no fuera bloqueadas.

Implementación de Pruebas Unitarias Automatizadas

Para garantizar la calidad y estabilidad de la lógica de negocio, se implementaron pruebas unitarias automatizadas en la capa de servicios utilizando **JUnit 5** y el framework de simulación **Mockito**.

Estrategia de Pruebas con Mockito

El objetivo de las pruebas unitarias es aislar el fragmento de código a evaluar. Al probar la clase ShoppingCartService, se utilizó la anotación @Mock para crear un doble simulacro de la interfaz ShoppingCartRepository. Esto evita que las pruebas modifiquen la base de datos real en MySQL, asegurando que las operaciones de prueba como vaciar un carrito, se ejecuten en un entorno controlado y seguro en memoria.

Estructura de la Prueba (Patrón AAA): La prueba diseñada para el método vaciarCarrito sigue el estándar de la industria AAA (Arrange, Act, Assert):

1. **Arrange (Preparar):** Se define un usuario simulado (usuario123) como parámetro de entrada.
2. **Act (Ejecutar):** Se invoca el método vaciarCarrito() de la capa de servicios.
3. **Assert (Verificar):** A través del método Mockito.verify(), el sistema audita internamente y comprueba que la lógica del servicio invocó de manera exitosa y exacta (1 sola vez) a la instrucción SQL dinámica deleteByCodUsuario del repositorio subyacente.

Esta implementación valida que el flujo de datos desde el servicio hasta la capa DAO opera de forma determinista y predecible.

Con esto, tienes tu documento estructurado cubriendo exactamente lo que pide la evaluación: justificación del entorno, diagramas UML, Mockups, y pruebas unitarias con JUnit 5 y Mockito.

Explicaciones más relevantes

Clase “CorsConfig”

La clase llamada CorsConfig en Spring Boot fue creada porque @ CORS, dio problemas. "CORS es una medida de seguridad de los navegadores web. El problema ocurrió porque mi frontend de Angular corría en el puerto 4200 y mi backend de Spring Boot en el puerto 8080. Cuando puertos distintos intentan comunicarse, el navegador bloquea la petición (especialmente los POST) por sospecha de ataque. Se resuelve en el backend mediante la clase CorsConfig porque es el servidor (Spring Boot) quien tiene que darle permiso explícito al navegador, enviando una cabecera HTTP que diga 'sí acepto peticiones del puerto 4200'."

El flujo de datos De Angular a MySQL

Para explicar acerca del flujo de la estructura y las capas, lo representa muy bien el echo de presionar el botón 'Agregar al carrito' en Angular o en el navegador visto en la página principal en donde se encuentra el catálogo de productos, esta orden llega a la base de datos**,** lo que significa que el flujo consta de 4 pasos; Primero, en Angular, el ProductoService usa HttpClient para enviar una petición POST con un objeto JSON, que lleva el ID del producto y la cantidad, segundo, la petición entra al puerto 8080 y la recibe el ShoppingCartController en Spring Boot, el cual usa un DTO (Data Transfer Object)

llamado ItemRequest para leer ese JSON, tercero, el controlador le pasa esos datos a la capa lógica (ShoppingCartService), y finalmente, el servicio utiliza la interfaz ShoppingCartRepository, la cual usa Spring Data JPA para inyectar automáticamente el registro en la tabla de MySQL."

El error 404 (Endpoints)

Como ejemplo, tuve el problema que en Angular se envió la petición POST, el servidor claro que ya estaba encendido, pero la URL en Angular tiene al revés la petición esto era algo que el controlador de Java no esperaba. Entonces lo que ocurrió fue que el servidor rechazo la petición y devolvió un error HTTP 404 (Not Found). Esto me sucedió durante el desarrollo: Spring Boot estaba configurado para recibir el JSON en el 'cuerpo' de la petición en la ruta /api/carrito/usuario123/agregar, pero Angular estaba enviando el ID en la URL /agregar/usuario123. El 404 significa que la conexión al puerto 8080 fue exitosa, pero Spring Boot no encontró ninguna ruta exacta que coincidiera con la firma de ese método."

Manual del Usuario

Manual de Ejecución y Despliegue Local

Para que el sistema pueda ser ejecutado correctamente desde cualquier entorno, se deben seguir estos pasos de levantamiento, pueden ser con un entorno Linux como Ubuntu o Windows con los puertos 8080 y 4200 liberados:

1. Base de Datos (MySQL)

- Asegurar que el servicio de MySQL esté activo en el puerto 3306 ya que este es el puerto por defecto en que se levanta MySQL.
- La base de datos ya está pre configurada, verificar que las credenciales en el archivo application.properties de Spring Boot coincidan con las del sistema anfitrión.

2. Backend (Spring Boot)

- Abrir la carpeta del backend en el IDE IntelliJ.
- Permitir que Maven descargue las dependencias del archivo pom.xml, normalmente el IDE hace esto de forma automática al abrir la carpeta del proyecto.

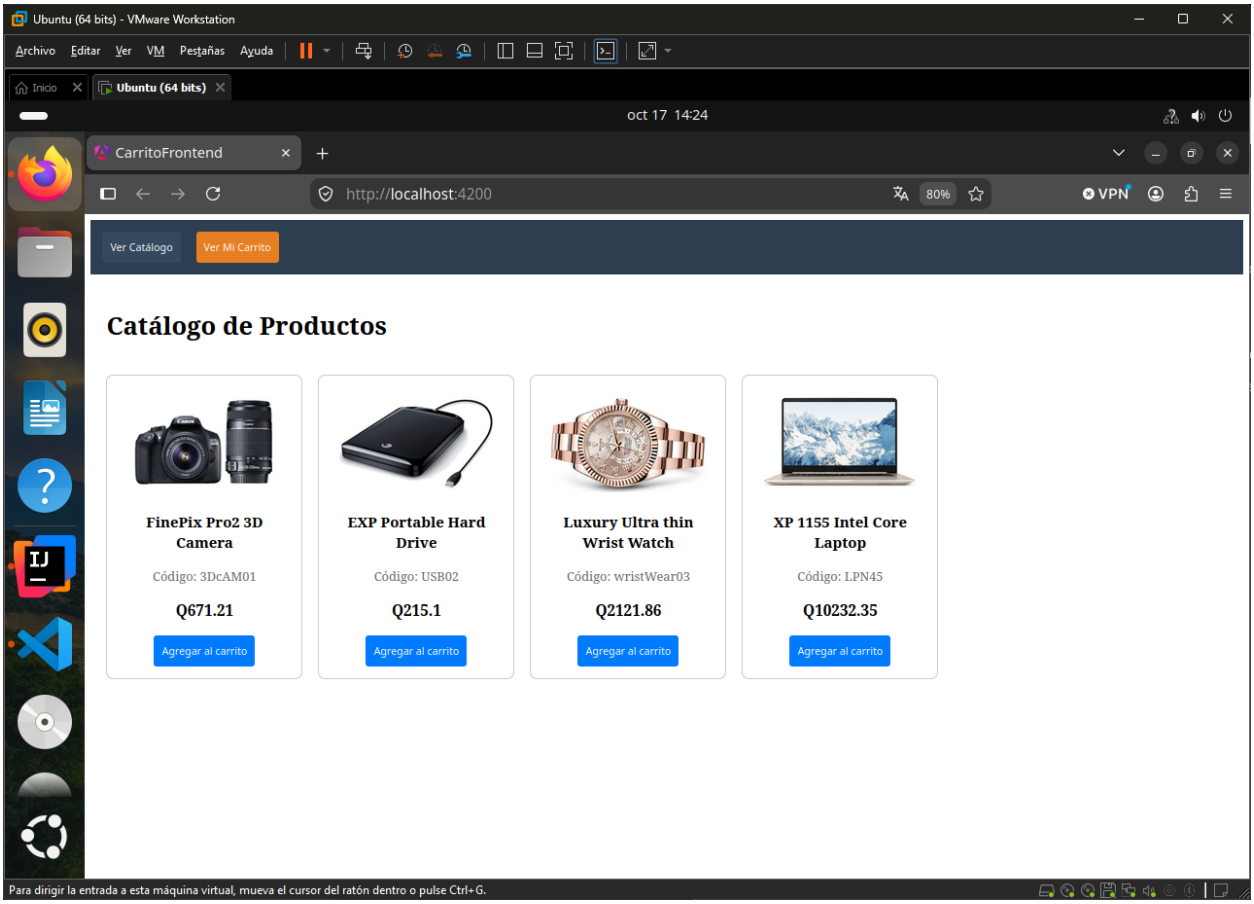
- Ejecutar la clase principal ApiApplication.java para levantar el servidor embebido de Tomcat en http://localhost:8080.

3. Frontend (Angular)

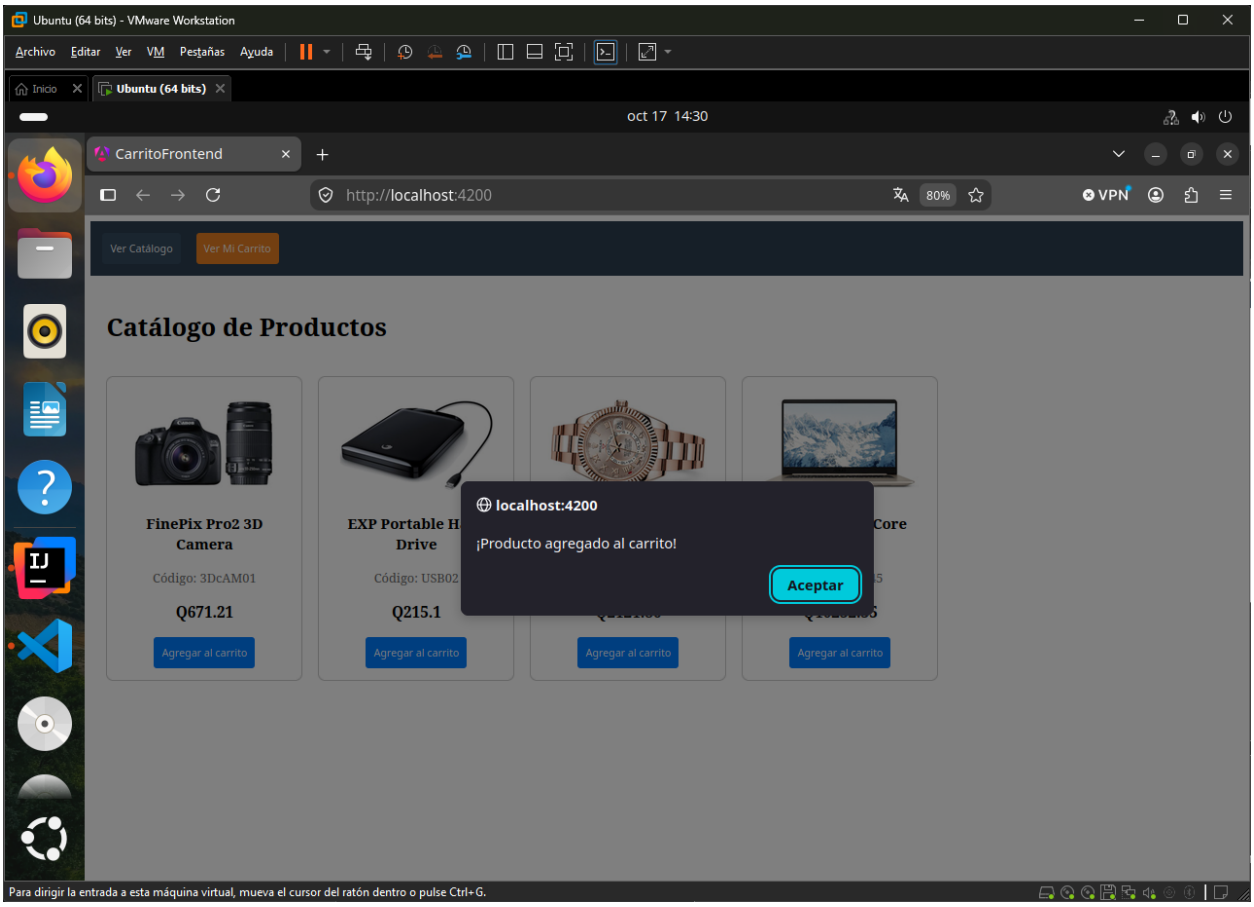
- Abrir una terminal en la carpeta en donde se encuentre: carrito-frontend.
- Ejecutar el comando npm install para restaurar los módulos de Node, en caso de ser necesario.
- Levantar el servidor de desarrollo ejecutando ng serve -o, lo cual abrirá automáticamente el sistema en http://localhost:4200, en el navegador por defecto que este configurado en el sistema operativo.

En el navegador

Cuando se habrá la pagina y muestra el catalogo de los productos, en la parte superior izquierda están los botones de “Ver Catalogo” y “Ver Mi carrito”



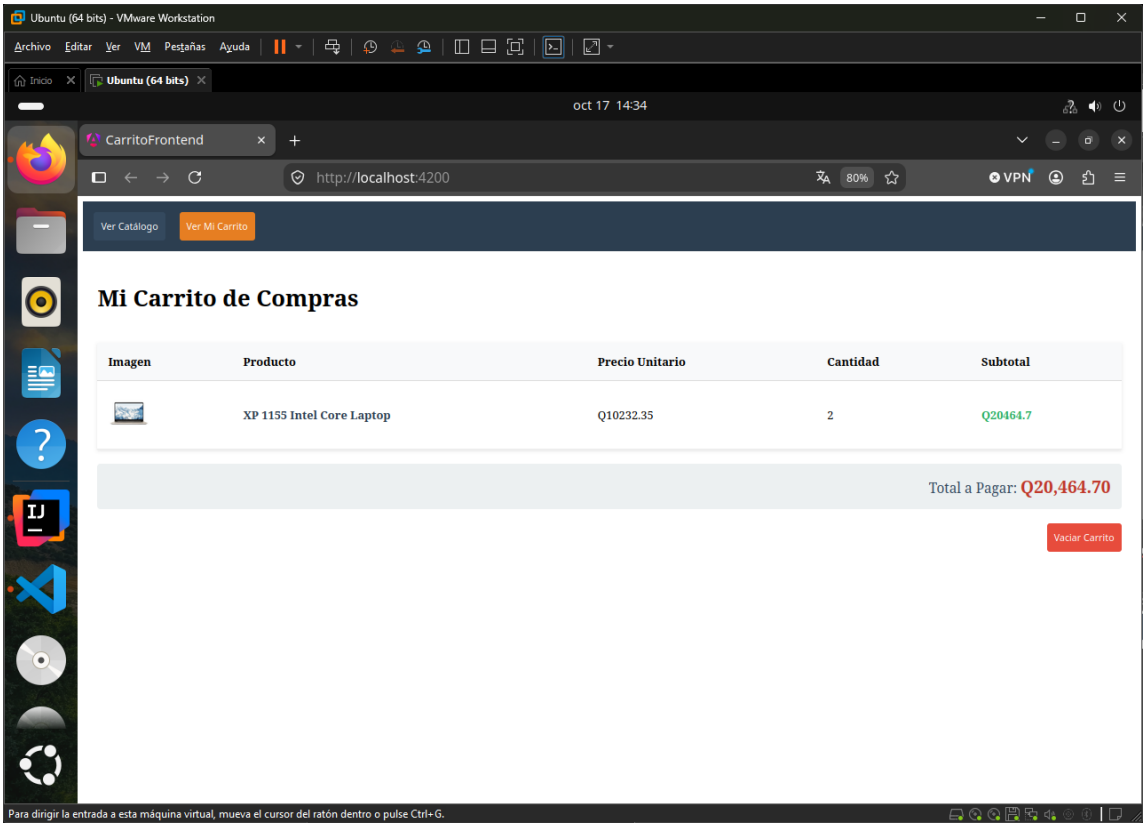
Dentro del Catálogo de Productos, puede dar click sobre el botón “Agregar al carrito”, que se encuentra debajo de cada producto. ¡Al pulsarlo se abrirá un cuadro de alerta en el cual le indica al usuario “! ¡Producto agregado al carrito!”, como se muestra en la siguiente imagen.



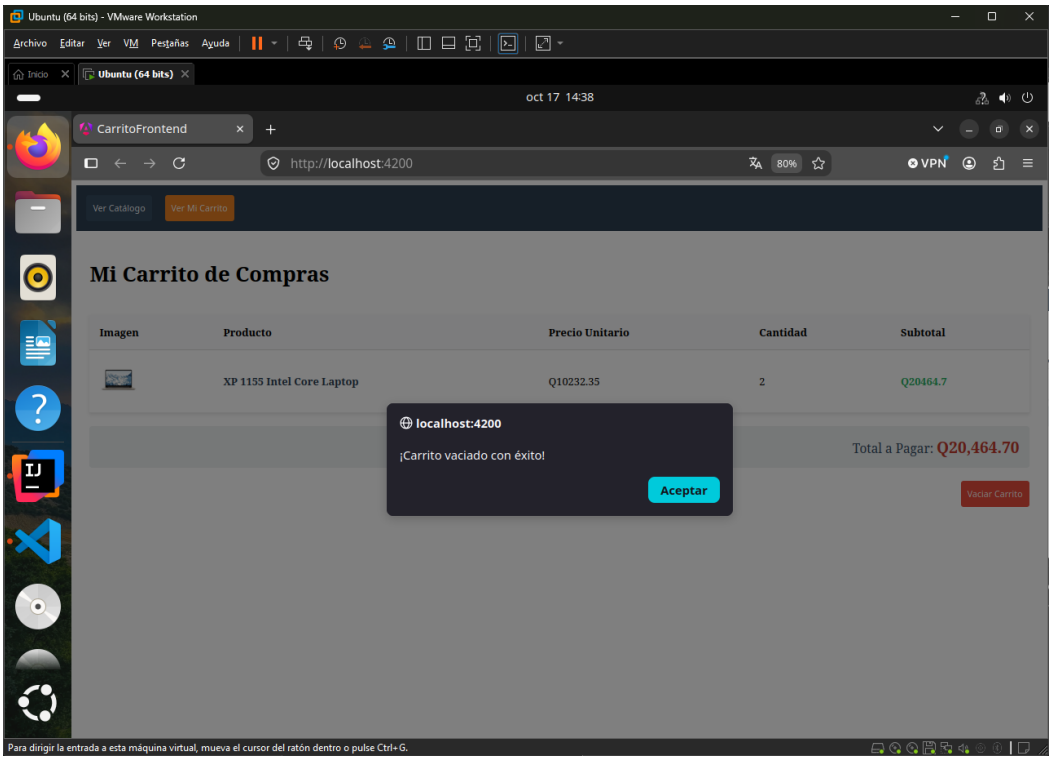
Para ver los productos que se han agregado, lo puede hacer dando click sobre el botón “Ver Mi Carrito”, en la parte superior izquierda de la página web.



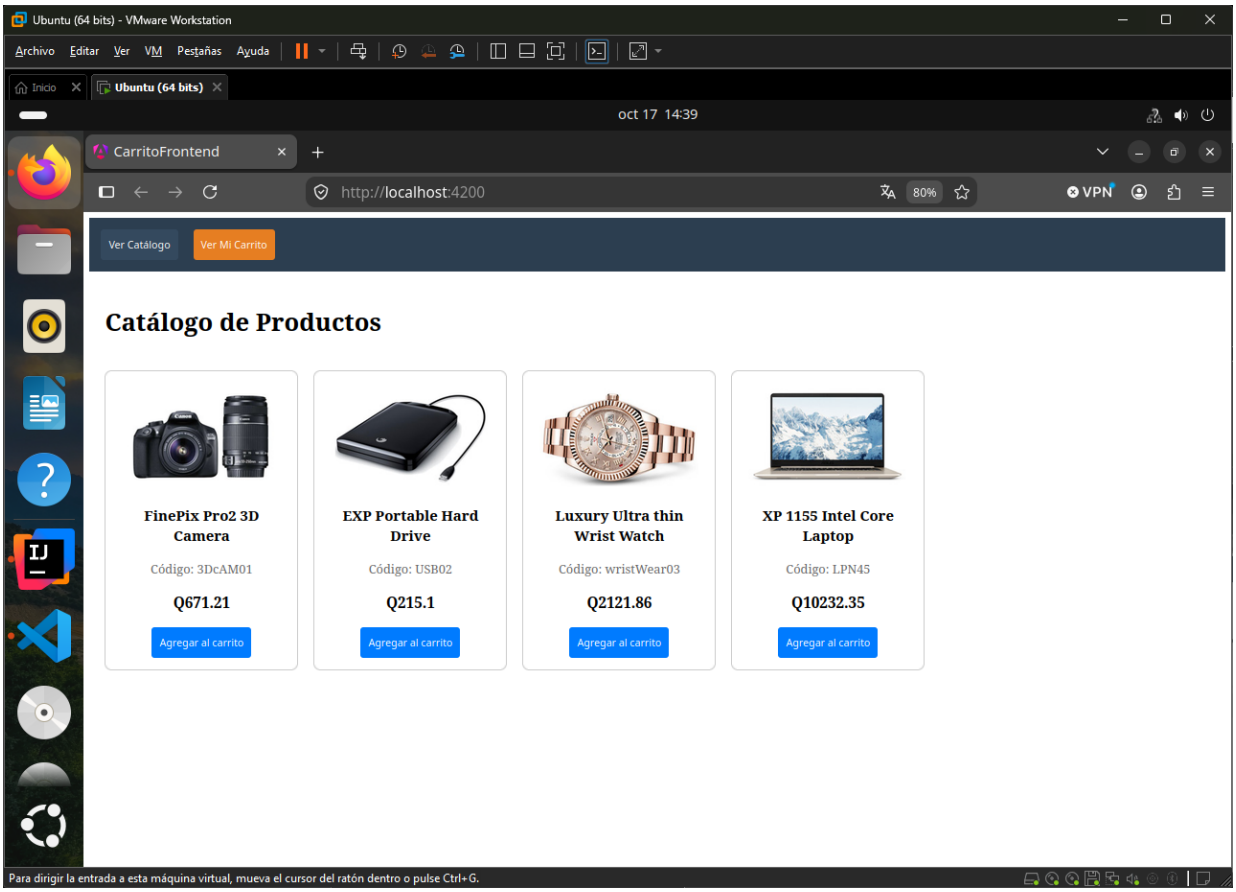
Una vez dentro de la pantalla de “Ver Mi Carrito”, aparecerá el producto seleccionado su precio, la cantidad de productos y el total a pagar.



Para vaciar el carrito de compras solo basta con hacer click sobre el botón “Vaciar Carrito”, esto le dará un cuadro de alerta con el mensaje “!Carrito vaciado con éxito!”



Al darle el botón de “Aceptar”, le devolverá de nuevo al Catalogo de Productos



Conclusión

Se logró migrar una arquitectura monolítica echa en PHP a una arquitectura distribuida Frontend y Backend por separado. Esto garantiza que si en el futuro se desea crear una aplicación móvil, la API de Spring Boot podrá reutilizarse sin modificar una sola línea de código en el servidor, además que esta separación es mucho más limpia, quizá más trabajosa, pero el código queda más escalable para su comprensión.

A lo largo de la virtualización del entorno de desarrollo en mi maquina virtual de Linux Ubunto por VMware, me permitió diagnosticar y resolver cuellos de botella en la memoria RAM ocasionados por la ejecución simultánea de los servidores de desarrollo, en un principio creí que la máquina virtual se había volcado, ya que solo me mostraba una consola negra y no había nada de interacción grafica más que solamente comandos por la terminal de Ubunto. Resolviendo todo esto con el aumento de los recursos de memoria RAM, espacio en DiscoLo, y aumentando el procesador, esto subraya la importancia de monitorear el consumo de hardware en aplicaciones Full-Stack.

Se resolvió exitosamente la restricción de seguridad de navegadores (CORS) mediante una configuración global en Spring Boot, permitiendo el cruce de datos seguro entre distintos puertos sin comprometer la integridad de las peticiones RESTful. Esto permition la completa comunicación entre el Frontend y el backend, y sobre todo la persistencia de datos en la Base de Datos.

Por ultimo debo añadir que la migración de un lenguaje como lo es PHP, de cierta manera no fue tan fácil, porque primero se debía de comprender como se comunicaba el navegador con el código, y también la forma en que se comunicaba hacia la base de datos.

Referencias y Webgrafía

- **Documentación Oficial de Spring Boot y Spring Data JPA:** Utilizada para comprender la inyección de dependencias, la configuración de controladores REST y la abstracción del patrón DAO. *https://spring.io/projects/spring-boot*, Empezando | Construcción de un servicio web RESTful, Empezando | Consumir un servicio web RESTful, Empezando | Acceso a datos con MySQL

- **Documentación Oficial de Angular:** Referencia principal para la implementación de la nueva arquitectura Standalone, el uso de la directiva de control de flujo @if y la configuración del HttpClient global. *https://angular.dev/*, *Control flow - @for • Angular*, *Property Binding • Angular*, *Event handling • Angular*, *Optimizing images • Angular*, *Define a route • Angular*, *Forms • Angular*, *Creating an injectable service • Angular*, *Injecting a dependency • Angular*

- **Resolución de Políticas CORS:** Guías técnicas y foros de arquitectura web para comprender el ciclo de peticiones HTTP, el bloqueo de navegadores y la configuración del WebMvcConfigurer en Java. Intercambio de recursos de origen cruzado (CORS) - HTTP | MDN, Errores CORS - HTTP | MDN, ¿Qué es CORS? - Explicación del uso compartido de recursos entre orígenes - AWS, Empezando | Habilitación de solicitudes de origen cruzado para un servicio web RESTful

- **Pruebas Unitarias con JUnit 5 y Mockito:** Referencia técnica para la creación de entornos simulados (Mocks) que permiten verificar la capa de servicios sin afectar la persistencia de la base de datos real. Cómo usar Junit y Mockito para pruebas unitarias en Java - Comunidad DEV